

UNIVERSIDAD DE TALCA
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA CIVIL EN COMPUTACIÓN

SmartHome IoT: Seguridad, LLM Local y Agente Autónomo

Informe Técnico — Unidad 2 (Equipo 69)

Asignatura: Internet de las Cosas
Profesor: Ricardo Pérez
Fecha: Junio 2026

Integrantes:

Mariano Muñoz R. `mariano.munoz@alumnos.utalca.cl`
Cristian Ruiz I. `cristian.ruiz@alumnos.utalca.cl`
Camilo Fuentes G. `camilo.fuentes@alumnos.utalca.cl`

23 de junio de 2026

Índice

1. Introducción	4
2. Infraestructura y Seguridad MQTT	5
2.1. Arquitectura General	5
2.2. Configuración TLS	6
2.3. Generación de Certificados	6
2.4. Autenticación y Control de Acceso (ACL)	6
2.5. Consideración de Seguridad: Puerto 1883	7
3. Actualización de Firmware con Soporte TLS	8
3.1. ESP32-CAM: Conexión Segura	8
3.2. Externalización de Credenciales	8
3.3. Lógica de Reconexión	8
4. LLM Local e Integración	9
4.1. Ollama + phi3:mini	9
4.2. LLM Gateway (FastAPI)	9
4.3. Prompts Dinámicos	10
4.4. Manejo de Errores y Fallback Determinístico	10
5. Servidor MCP (Model Context Protocol)	11
5.1. Tools Disponibles	11
5.2. Flujo de Comunicación	11
6. Agente Autónomo LangGraph	12
6.1. Arquitectura del Grafo	12
6.2. Descripción de Nodos	12
6.3. Ruteo Condicional	13
6.4. Bypass Crítico (Sin LLM)	13
7. Bitácora de Problemas Detectados y Soluciones	14
8. Ensayo Crítico: LLM vs. Control Determinista	15
8.1. Introducción	15
8.2. Tabla Comparativa	15
8.3. Análisis	15
8.4. Conclusión	16
9. Análisis de Vulnerabilidad IoT	17
9.1. Introducción	17
9.2. Vulnerabilidad 1: MKR1000 sin TLS	17
9.3. Vulnerabilidad 2: Healthcheck sin autenticación	17
9.4. Vulnerabilidad 3: Certificados autofirmados	17
9.5. Vulnerabilidad 4: Sin rate limiting en API	18

9.6. Vulnerabilidad 5: Exposición de puertos Docker	18
9.7. Plan de Mitigación Priorizado	18
10.Conclusiones	19

Índice de figuras

1. Arquitectura del sistema en Unidad 2, mostrando los componentes de seguridad, LLM y agente autónomo. 5
2. Diagrama de flujo del agente LangGraph con 15 nodos y rutas de decisión. 12

Índice de tablas

1. Archivos de certificados TLS generados. 6
2. Usuarios MQTT y sus permisos ACL. 7
3. Lógica del fallback determinístico cuando el LLM no está disponible. . . 10
4. Tools MCP disponibles para el agente LangGraph. 11
5. Nodos del StateGraph LangGraph. 12
6. Bitácora de problemas del LLM y soluciones aplicadas. 14
7. Comparación entre control determinista y basado en LLM. 15

1. Introducción

En la Unidad 1 del proyecto se estableció la infraestructura base de comunicación MQTT con un broker Mosquitto en texto plano, la integración de sensores/actuadores con Arduino MKR1000 y ESP32-CAM, y un sistema de reconocimiento facial para control de acceso. Sobre esa base, la Unidad 2 añade tres pilares fundamentales:

1. **Seguridad en las comunicaciones:** Migración de MQTT texto plano a TLS con autenticación por usuario/contraseña y políticas de control de acceso (ACL) granulares para cada dispositivo.
2. **LLM Local:** Integración de un modelo de lenguaje pequeño (phi3:mini vía Ollama) como cerebro del hogar, capaz de analizar contexto sensorial y decidir acciones.
3. **Agente Autónomo:** Implementación de un agente LangGraph con 15 nodos que monitoriza el hogar, evalúa umbrales, consulta el LLM cuando es necesario y ejecuta acciones de forma segura a través de un servidor MCP.

Este informe detalla la arquitectura, implementación y resultados de cada uno de estos componentes, incluyendo un análisis de vulnerabilidades y una reflexión crítica sobre el uso de LLMs en entornos de producción IoT.

2. Infraestructura y Seguridad MQTT

2.1. Arquitectura General

La Figura 1 muestra la arquitectura completa de la Unidad 2. El broker Mosquitto actúa como bus central de comunicación, con dos listeners diferenciados:

- **Puerto 8883 (TLS):** Para servicios internos Docker (Node-RED, Backend, LLM Gateway, Digital Twin, MCP Server).
- **Puerto 1884 (plain + auth):** Para placas embebidas (MKR1000, ESP32-CAM) cuyo hardware no soporta TLS moderno.

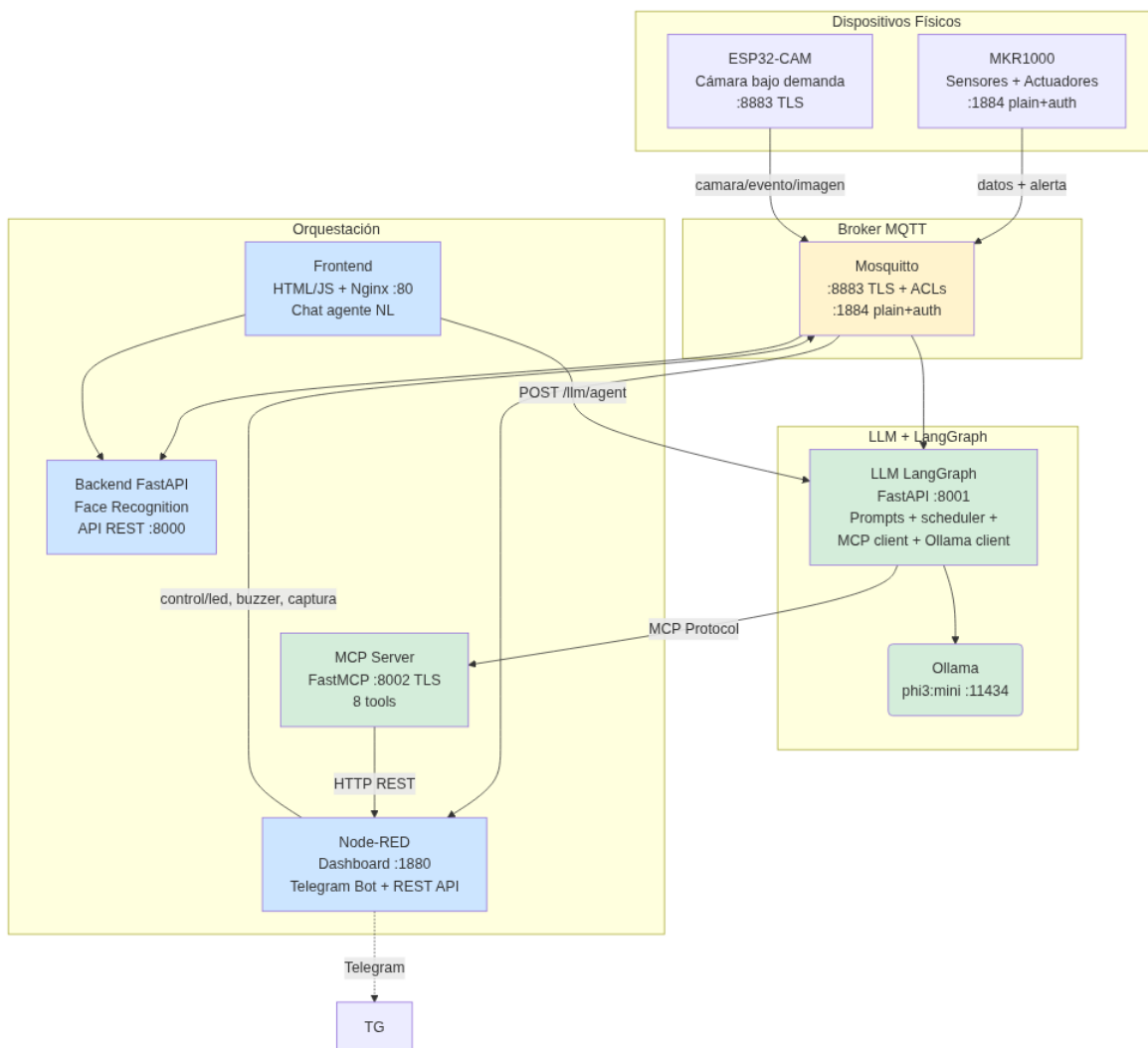


Figura 1: Arquitectura del sistema en Unidad 2, mostrando los componentes de seguridad, LLM y agente autónomo.

2.2. Configuración TLS

El archivo `mosquitto.conf` define tres listeners con diferentes niveles de seguridad. El Listado 1 muestra la configuración completa.

```

1 # TLS listener — servicios internos
2 listener 8883 0.0.0.0
3 cafile /mosquitto/certs/ca.crt
4 certfile /mosquitto/certs/server.crt
5 keyfile /mosquitto/certs/server.key
6 require_certificate false
7 password_file /mosquitto/config/passwd
8 acl_file /mosquitto/config/acl.conf
9 allow_anonymous false
10
11 # Non-TLS listener — placas embebidas
12 listener 1884 0.0.0.0
13 password_file /mosquitto/config/passwd
14 acl_file /mosquitto/config/acl.conf
15 allow_anonymous false
16
17 # Healthcheck listener — localhost only
18 listener 1883 127.0.0.1
19 allow_anonymous true

```

Listing 1: Configuración de Mosquitto con TLS y ACLs por listener.

2.3. Generación de Certificados

Los certificados TLS se generan mediante el script `generate-certs.sh`, que crea una Autoridad Certificadora (CA) autofirmada con validez de 365 días y firma los certificados del servidor y del MCP Server. El script `sync-ca-to-firmware.sh` sincroniza el CA cert con el firmware de la ESP32-CAM para que pueda validar la conexión TLS.

Un aspecto crítico del diseño es que la ESP32-CAM requiere sincronización NTP para validar la fecha del certificado, ya que el chip ESP32 no tiene RTC con batería de respaldo.

Tabla 1: Archivos de certificados TLS generados.

Archivo	Rol	Ubicación
<code>ca.crt</code> / <code>ca.key</code>	Autoridad Certificadora	<code>deploy/mosquitto/certs/</code>
<code>server.crt</code> / <code>server.key</code>	Certificado del broker	<code>deploy/mosquitto/certs/</code>
<code>mcp-server.crt</code> / <code>mcp-server.key</code>	Certificado MCP Server	<code>deploy/mosquitto/certs/</code>

2.4. Autenticación y Control de Acceso (ACL)

Se implementaron seis usuarios MQTT, cada uno con permisos estrictamente limitados a los tópicos necesarios para su función. El Cuadro 2 resume los perfiles y sus

permisos.

Tabla 2: Usuarios MQTT y sus permisos ACL.

Usuario	Permisos	Rol
mkr1000-equip069	Write: datos, alerta Read: control/led, control/led-puerta	Publicar sensores Recibir comandos
esp32cam-equip069	Write: camara/evento, camara/imagen Read: camara/captura	Publicar imágenes Recibir órdenes
backend-equip069	Write: acceso/*, control/led-puerta Read: camara/imagen, camara/url	Control acceso Procesar imágenes
nodered-smarthome	Write: control/*, camara/captura Read: smarthome/+/datos, llm/decision	Orquestador Monitoreo
llm-gateway-equip069	Write: llm/decision (sin read)	Solo publicar decisiones
digital-twin-equip069	Read: smarthome/equip069/# Write: prediccion/*	Estado completo Publicar predicciones

Nótese que el usuario `llm-gateway-equip069` NO tiene permisos de escritura en ningún tópico de control (`control/led`, `control/led-puerta`, etc.). Esto es intencional: el LLM nunca publica directamente comandos a los actuadores. Node-RED actúa como gatekeeper, recibiendo las decisiones del LLM y decidiendo si ejecutarlas o no.

```

1 # LLM Gateway — reasoning decisions only (publish-only, no subscribe)
2 user llm-gateway-equip069
3 topic write smarthome/equip069/llm/decision

```

Listing 2: Extracto de `acl.conf` mostrando las restricciones del LLM Gateway.

2.5. Consideración de Seguridad: Puerto 1883

Si bien la especificación solicita deshabilitar completamente el puerto 1883 en texto plano, en nuestra implementación este puerto permanece activo pero restringido exclusivamente a `127.0.0.1` (localhost) para permitir el healthcheck de Docker sobre Mosquitto. Dado que no es accesible desde la red externa y no requiere autenticación, representa un riesgo mínimo controlado. Esta decisión de diseño se tomó para mantener la capacidad de orquestación sin comprometer la seguridad perimetral.

3. Actualización de Firmware con Soporte TLS

3.1. ESP32-CAM: Conexión Segura

El firmware de la ESP32-CAM se actualizó para usar `WiFiClientSecure` con certificado CA embebido. El flujo de conexión es:

1. Conectar a WiFi.
2. Sincronizar hora vía NTP (necesario para validar la fecha del certificado TLS).
3. Configurar `WiFiClientSecure.setCACert()` con el CA cert del proyecto.
4. Conectar al broker Mosquitto en puerto 8883.
5. Publicar evento `camara/evento` con `{"status": "camara_lista"}`.

El CA cert se inyecta automáticamente en el firmware mediante el script `sync-ca-to-firmware.sh`, que extrae el certificado del contenedor Docker de Mosquitto y lo inserta en el archivo `secrets.h` de la ESP32-CAM.

3.2. Externalización de Credenciales

Ambos firmwares utilizan el patrón `secrets.h.example + secrets.h` (en `.gitignore`) para mantener las credenciales fuera del repositorio. Los templates contienen valores placeholder que el desarrollador debe reemplazar antes de flashear.

3.3. Lógica de Reconexión

Ambos firmwares implementan reconexión automática con backoff exponencial:

- **MKR1000** (`mqtt_manager.cpp`): Reintenta conexión MQTT cada 5 segundos si se pierde la conexión. Reintenta WiFi cada 10 segundos si se cae la red.
- **ESP32-CAM** (`mqtt_bridge.cpp`): Similar, con reintentos cada 5 segundos para MQTT y 10 segundos para WiFi.

4. LLM Local e Integración

4.1. Ollama + phi3:mini

Ollama se ejecuta en un contenedor Docker con el modelo `phi3:mini` (3.8B parámetros). Se configuró con:

- `OLLAMA_KEEP_ALIVE=-1`: Mantiene el modelo cargado en memoria para evitar tiempos de carga fría.
- `OLLAMA_NUM_THREADS=6`: Paralelismo para CPU (sin GPU).
- Límite de memoria: 4 GB.

4.2. LLM Gateway (FastAPI)

El LLM Gateway es un microservicio FastAPI en el puerto `:8001` que centraliza todas las interacciones con Ollama. Sus responsabilidades incluyen:

- **Decisión de seguridad:** Toma el contexto actual de sensores (temperatura, humedad, gas, ruido), construye un prompt dinámico y consulta a Ollama para determinar el nivel de alerta.
- **Forzado de formato JSON:** La petición HTTP a Ollama incluye `"format": "json"` para obligar al modelo a devolver una estructura parseable. El Listado 3 muestra la configuración en `ollama_client.py`.
- **Gestión de errores:** Timeout estricto de 3 segundos para decisiones, con retry exponencial y fallback determinístico si el LLM no responde.
- **Scheduler del agente:** Ejecuta el ciclo autónomo del LangGraph Agent cada 30 segundos.
- **Endpoint REST:** Expone `POST /llm/agent` para consultas del usuario desde el frontend.

```
1 payload = {
2     "model": self.model,
3     "prompt": prompt,
4     "stream": False,
5 }
6 if format_json:
7     payload["format"] = "json" # Obliga a Ollama a devolver JSON
8 if system_prompt is not None:
9     payload["system"] = system_prompt
```

Listing 3: Forzado de formato JSON en las peticiones a Ollama.

4.3. Prompts Dinámicos

El prompt de decisión se construye en tiempo real inyectando los valores actuales de los sensores. El Listado 4 muestra el sistema de prompts.

```

1 system_prompt = (
2     "Sos el agente de seguridad de una casa inteligente (SmartHome
3     equipo69).\n"
4     "Tu tarea es analizar los datos de sensores y determinar el nivel
5     de alerta.\n\n"
6     "Niveles de alerta:\n"
7     "- critico: gas > 1020 ppm o temperatura > 30 °C (emergencia)\n"
8     "- alto: gas > 800 ppm o temperatura > 28 °C (riesgo significativo)
9     )\n"
10    "- medio: gas > 400 ppm o ruido > 85 dB (precaución)\n"
11    "- bajo: valores levemente fuera de rango\n"
12    "- info: todo normal\n\n"
13    "Respondé SOLO con un JSON en este formato exacto:\n"
14    '{"nivel": "critico|alto|medio|bajo|info", '
15    '"razonamiento": "explicación en español", '
16    '"acciones": [{"tool": "nombre", "args": {...}}], '
17    '"confidence": 0.0-1.0}'
18 )

```

Listing 4: Constructor de prompts dinámicos para decisión de seguridad.

4.4. Manejo de Errores y Fallback Determinístico

El sistema implementa una estrategia de tolerancia a fallos en cascada para el LLM:

1. **Timeout:** La llamada a Ollama tiene un timeout de 3 segundos envuelto en `asyncio.wait_for`.
2. **Retry exponencial:** `httpx.AsyncClient` con backoff $2^{attempt}$ segundos (1s, 2s, 4s, ...).
3. **Fallback determinístico:** Si el LLM falla o devuelve JSON inválido, se llama a `_deterministic_fallback()` para aplicar reglas fijas (Cuadro 3).

Tabla 3: Lógica del fallback determinístico cuando el LLM no está disponible.

Condición	Acción
Gas > 1020 ppm o Temp > 30°C	Nivel crítico: LED alerta + Telegram
Gas > 800 ppm o Temp > 28°C	Nivel alto: solo Telegram
Gas > 400 ppm	Nivel medio: solo Telegram
Valores normales	Nivel normal: sin acciones

5. Servidor MCP (Model Context Protocol)

El MCP Server es un servicio Python independiente que implementa el protocolo MCP sobre Streamable HTTP con TLS en el puerto :8002. Expone 8 herramientas (tools) que encapsulan las operaciones del hogar inteligente.

5.1. Tools Disponibles

Tabla 4: Tools MCP disponibles para el agente LangGraph.

Tool	Tipo	Descripción
<code>get_sensor_state</code>	Info	Temperatura, humedad, gas, sonido actuales
<code>get_system_status</code>	Info	Estado lógico de LED y alertas
<code>query_history</code>	Info	Historial CSV con filtro temporal
<code>activate_led_alerta</code>	Acción	Encender/apagar LED de alerta
<code>activate_led_puerta</code>	Acción	Encender/apagar LED de puerta
<code>send_notification</code>	Acción	Enviar mensaje por Telegram
<code>silence_alerts</code>	Acción	Apagar todas las alertas
<code>trigger_camera</code>	Acción	Disparar captura de cámara

5.2. Flujo de Comunicación

El MCP Server nunca se conecta directamente al broker MQTT. En su lugar, traduce llamadas MCP a peticiones HTTP REST hacia Node-RED, que es el único componente autorizado para publicar en tópicos de control MQTT:

```
LangGraph → MCP Sidecar (:8002, TLS)
  → HTTP → Node-RED REST (:1880)
  → MQTT → dispositivos
```

Este diseño garantiza que el LLM (o cualquier cliente MCP) nunca tenga acceso directo a los actuadores físicos, cumpliendo con el principio de mínimo privilegio.

6. Agente Autónomo LangGraph

6.1. Arquitectura del Grafo

El agente se implementa como un StateGraph de LangGraph con 15 nodos interconectados mediante aristas condicionales. La Figura 2 muestra el diagrama de flujo completo.

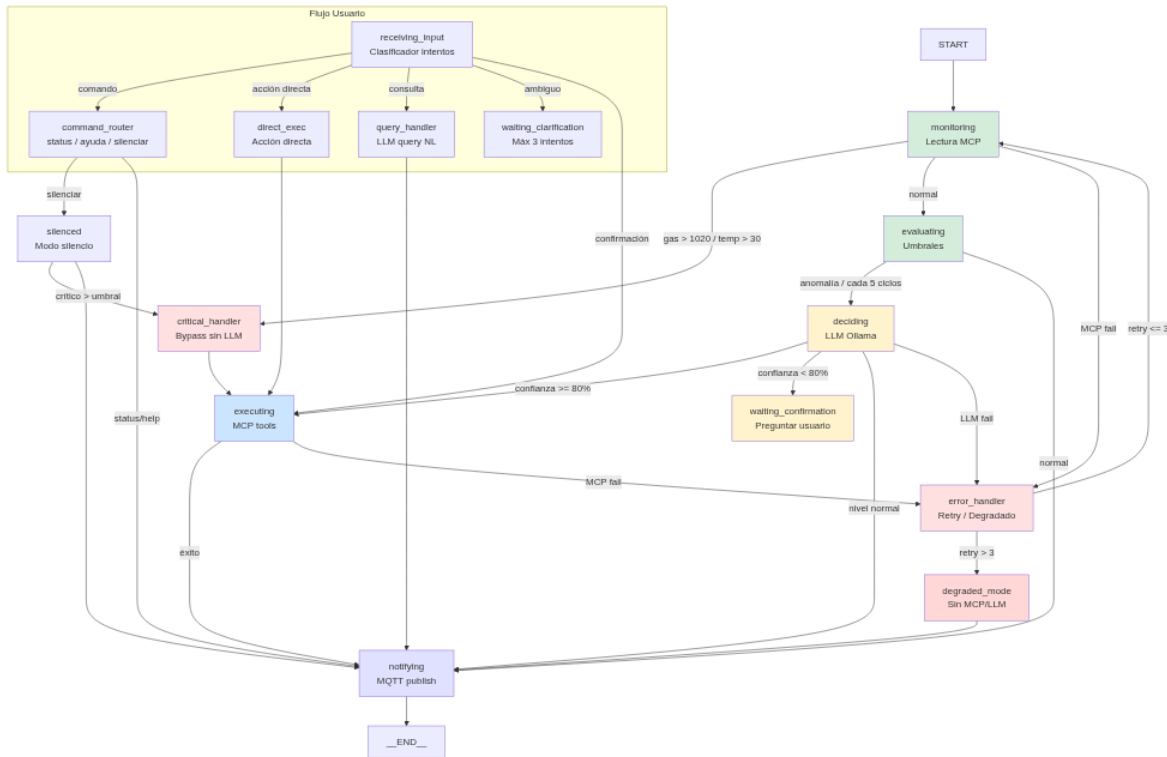


Figura 2: Diagrama de flujo del agente LangGraph con 15 nodos y rutas de decisión.

6.2. Descripción de Nodos

Tabla 5: Nodos del StateGraph LangGraph.

Nodo	Entrada	Función
monitoring	MCP sensors	Lee datos de sensores vía MCP. Detecta staleness.
evaluating	Sensor data	Evalúa umbrales: anomalía, tendencia ascendente.
critical_handler	Umbral crítico	Bypass crítico SIN LLM: plan fijo de acciones.
deciding	Contexto sensores	Consulta a Ollama para decisión. Timeout 3s.
executing	Acciones plan	Ejecuta secuencialmente vía MCP.
notifying	Decisión	Prepara payload y publica a MQTT.

Tabla 5: Nodos LangGraph (continuación)

Nodo	Entrada	Función
<code>error_handler</code>	Error	Retry (máx 3) o modo degradado.
<code>degraded_mode</code>	Error persistente	Desactiva MCP, notifica al usuario.
<code>receiving_input</code>	Usuario texto	Clasifica intent con reglas + LLM.
<code>command_router</code>	Intento	/status, /ayuda, silenciar.
<code>query_handler</code>	Consulta NL	Responde preguntas con LLM + datos MCP.
<code>direct_exec</code>	Acción directa	Keywords → herramienta MCP.
<code>waiting_confirmation</code>	Baja confianza	Pide confirmación al usuario.
<code>waiting_clarification</code>	Intento ambiguo	Pide reformular (<3 intentos).
<code>silenced</code>	Modo silencio	Suprime alertas no críticas.

6.3. Ruteo Condicional

El flujo del agente utiliza aristas condicionales que evalúan el estado actual para decidir el siguiente nodo. Los puntos de decisión más relevantes son:

- **Post-monitoreo:** Si $\text{gas} > 1020 \text{ ppm}$ o $\text{temp} > 30^\circ\text{C}$, deriva a `critical_handler` (bypass completo del LLM). Si MCP no responde, va a `error_handler`.
- **Post-evaluación:** Si hay anomalía o tendencia ascendente, consulta al LLM. Si es el 5to ciclo normal, también consulta al LLM para reporte razonado. Si es normal entre ciclos, notifica directamente sin gastar tokens.
- **Post-decisión:** Si $\text{confianza} \geq 80\%$ o hay acción crítica, ejecuta directo. Si $\text{confianza} < 80\%$ y hay acciones riesgosas, pide confirmación.
- **Post-error:** Si $\text{retry} \leq 3$, reintenta el ciclo. Si excede, entra en modo degradado.

6.4. Bypass Crítico (Sin LLM)

El nodo `critical_handler` es un bypass de emergencia que NO utiliza el LLM. Cuando se detectan valores críticos ($\text{gas} > 1020 \text{ ppm}$ o $\text{temp} > 30^\circ\text{C}$), ejecuta un plan fijo de acciones:

1. Activar LED de alerta.
2. Enviar notificación por Telegram.
3. Activar cámara para capturar evidencia.

Esto garantiza que incluso si el LLM está caído, timeout o dando respuestas incoherentes, el hogar responde adecuadamente a emergencias reales.

7. Bitácora de Problemas Detectados y Soluciones

Durante el desarrollo e integración del LLM y el agente autónomo, se enfrentaron diversos problemas. El Cuadro 6 resume los más significativos.

Tabla 6: Bitácora de problemas del LLM y soluciones aplicadas.

Problema	Descripción	Solución
Timeouts en phi3:mini	El modelo tardaba más de 30s en responder en CPU, causando timeouts en cascada.	Aumento progresivo de timeouts: classify 45s → 90s, chat 90s → 180s, endpoint 120s. Pre-warming del modelo al startup.
Clasificación errónea de intents	El LLM clasificaba “ver quién está en la puerta” como consulta en lugar de acción de cámara.	Reglas de key-words específicas (ver/mirar/fijarse/revisar → <code>trigger_camera</code>). Umbral de confirmación del 50 % para respuestas sí/no.
JSON mal formateado	phi3:mini devolvía JSON con markdown fences, campos faltantes o strings literales inválidos.	Implementación de <code>extract_json()</code> que limpia fences, busca primer/last brace, y retry con prompt de corrección.
Latencia en frío	Primera consulta tras startup tomaba > 60s por carga del modelo a memoria.	Pre-warming: <code>_prewarm_ollama()</code> envía un “ping” 2 segundos después del startup.
Falsos positivos en cámara vs. query	“Ve la temperatura” se clasificaba como acción de cámara por la palabra “ve”.	Reglas específicas: “ve” sin “cámara/puerta/foto” → query. “ver/mirar/fijarse” con contexto de cámara → acción.
Ollama no disponible	El contenedor Ollama tardaba en arrancar o se quedaba sin memoria.	Fallback determinístico (<code>_deterministic_fallback</code>) con umbrales fijos. Timeout de 3s con retry exponencial.
Ciclo de decisión lento	Consultar al LLM cada 30s consumía recursos innecesariamente en condiciones normales.	Skip de LLM para ciclos normales intermedios. Solo consulta al LLM en anomalías o cada 5 ciclos normales.

8. Ensayo Crítico: LLM vs. Control Determinista

8.1. Introducción

El sistema implementa ambos paradigmas de decisión en paralelo: control determinista clásico (bypass crítico, fallback, reglas de Node-RED) y decisión basada en LLM (phi3:mini vía LangGraph). Esta dualidad permite una comparación directa y honesta de sus fortalezas y debilidades en un entorno IoT de producción.

8.2. Tabla Comparativa

Tabla 7: Comparación entre control determinista y basado en LLM.

Dimensión	Control Determinista	LLM (phi3:mini)
Latencia	< 100 ms (if/else simple)	1-3 segundos en CPU
Predictibilidad	Siempre idéntico para mis- mas entradas	No determinista: puede va- riar entre ejecuciones
Mantenimiento	Umbrales en constantes configurables	Prompts requieren inge- niería y pruebas
Flexibilidad	Limitado a casos programa- dos	Puede manejar consultas no estructuradas
Consumo recursos	~0 MB RAM adicional	~3 GB RAM + CPU conti- nua
Tolerancia a fallos	Sin dependencias externas	Depende de Ollama + mo- delo
Capacidad expresiva	“LED alerta ON”	“Hay una anomalía de gas moderada, sugiero activar el LED y notificar al dueño”
Seguridad	100 % controlable vía ACLs	Riesgo de jailbreak, prompt injection

8.3. Análisis

Latencia: El control determinista es órdenes de magnitud más rápido. Una decisión con if/else toma microsegundos, mientras que phi3:mini requiere entre 1 y 3 segundos en CPU para generar una respuesta. Para aplicaciones de seguridad crítica (detección de gas), esta latencia es inaceptable.

Predictibilidad: El LLM es inherentemente no determinista. Un mismo contexto de sensores puede generar decisiones diferentes entre ejecuciones debido a la naturaleza proba- bilística del modelo. El control determinista siempre produce la misma salida para la misma entrada, lo que es esencial para auditorías de seguridad.

Flexibilidad: Aquí el LLM brilla. Un usuario puede preguntar “¿hace cuánto que no revisamos la puerta?” y el sistema responde en lenguaje natural. El control determinista requeriría programar explícitamente cada posible consulta.

Consumo de recursos: phi3:mini consume aproximadamente 3 GB de RAM y utiliza el 100 % de una CPU durante la inferencia. Para un sistema embebido o de bajos recursos, esto es prohibitivo. El control determinista funciona en un Arduino MKR1000 con 256 KB de RAM.

Tolerancia a fallos: El LLM introduce dependencias frágiles: el modelo debe estar cargado, Ollama debe responder, el tiempo de inferencia debe ser aceptable. Por el contrario, el control determinista funciona incluso si el contenedor Ollama está caído.

8.4. Conclusión

La arquitectura óptima es híbrida, que es exactamente lo que implementamos:

1. **Bypass determinista para emergencias:** Gas > 1020 ppm → actuar inmediatamente, sin LLM.
2. **LLM para análisis contextual:** Evaluación de tendencias, consultas en lenguaje natural, decisiones con múltiples variables.
3. **Fallback automático:** Si el LLM no responde en 3 segundos, el sistema cae a reglas deterministas.

El LLM no reemplaza al control determinista clásico. Lo complementa. En un entorno de producción IoT, la seguridad nunca debe depender exclusivamente de un modelo de lenguaje.

9. Análisis de Vulnerabilidad IoT

9.1. Introducción

Los sistemas IoT presentan vectores de ataque únicos debido a la heterogeneidad de hardware, las limitaciones de recursos y la naturaleza distribuida de las comunicaciones. Este análisis identifica vulnerabilidades en la arquitectura actual y propone mitigaciones.

9.2. Vulnerabilidad 1: MKR1000 sin TLS

Descripción: El Arduino MKR1000 utiliza WiFi101, cuyo stack TLS no es compatible con OpenSSL 3.x, por lo que se conecta al broker en texto plano (puerto 1884). Un atacante con acceso a la red local podría interceptar los datos de sensores o inyectar comandos falsos.

Riesgo: Medio-Alto. Requiere acceso a la red local, pero una vez dentro, el tráfico MQTT es completamente visible.

Mitigación:

- Autenticación por usuario/contraseña (no hay conexiones anónimas en 1884).
- ACLs restrictivas: MKR1000 solo puede escribir en `datos` y `alerta`, y leer en `control/*`.
- Segmentación de red VLAN para aislar dispositivos IoT.

9.3. Vulnerabilidad 2: Healthcheck sin autenticación

Descripción: El puerto 1883 en localhost no requiere autenticación. Aunque está ligado a 127.0.0.1, un atacante con ejecución de código en el host podría publicar mensajes arbitrarios.

Riesgo: Bajo. Requiere compromiso previo del host.

Mitigación:

- Restricción del listener a 127.0.0.1 (implementado).
- Eliminar el listener 1883 y usar `mosquitto_sub` con credenciales para healthcheck.

9.4. Vulnerabilidad 3: Certificados autofirmados

Descripción: La CA autofirmada no es verificable por una autoridad externa. Un atacante que comprometa el host podría reemplazar el CA cert y realizar un ataque MITM.

Riesgo: Medio. El CA cert reside en el sistema de archivos del host Docker.

Mitigación:

- Almacenamiento del CA cert con permisos restrictivos (modo 400).
- Firma de certificados por dispositivo con expiración.
- Monitoreo de integridad del CA cert mediante hashes.

9.5. Vulnerabilidad 4: Sin rate limiting en API

Descripción: Los endpoints REST de Node-RED (usados por MCP Server) no implementan rate limiting. Un atacante podría abusar del LLM Gateway para consumir recursos de Ollama.

Riesgo: Medio. Podría degradar el servicio.

Mitigación:

- Implementar rate limiting en Nginx o en el propio Node-RED.
- Timeout estricto de 3s en decisiones del LLM.

9.6. Vulnerabilidad 5: Exposición de puertos Docker

Descripción: Múltiples servicios exponen puertos al host (1880, 8000, 8001, 8002, 8003, 11434). Un atacante con acceso a la red local podría interactuar directamente con APIs internas.

Riesgo: Medio-Alto. Los puertos están abiertos en 0.0.0.0.

Mitigación:

- Usar redes Docker internas y solo exponer los puertos necesarios (80, 1880, 8883).
- Autenticación en todos los endpoints (JWT o API keys).
- TLS en MCP Server (:8002) y en frontend (:443).

9.7. Plan de Mitigación Priorizado

1. **Corto plazo:** Rate limiting en Nginx, restricción de puertos en docker-compose.yml, revisión de permisos de archivos de certificados.
2. **Mediano plazo:** Implementar autenticación en APIs REST de Node-RED y LLM Gateway, migrar healthcheck a listener autenticado.
3. **Largo plazo:** Evaluar hardware con soporte TLS 1.3 para MKR1000, implementar autenticación mutua (certificados de cliente) en ESP32-CAM.

10. Conclusiones

La Unidad 2 del proyecto SmartHome IoT ha demostrado que es posible integrar un modelo de lenguaje local (phi3:mini) en un sistema IoT de recursos limitados sin sacrificar la seguridad ni la confiabilidad.

Logros principales:

1. Se implementó una infraestructura de seguridad MQTT completa con TLS, autenticación por 6 usuarios y ACLs granulares, garantizando que cada dispositivo tenga solo los permisos necesarios.
2. El LLM Gateway centraliza las interacciones con Ollama, con manejo robusto de errores que incluye reintentos, timeouts y fallback determinístico.
3. El MCP Server expone 8 herramientas reutilizables de forma segura, manteniendo a Node-RED como gatekeeper único de los actuadores físicos.
4. El agente LangGraph con 15 nodos demostró ser capaz de monitorizar el hogar autónomamente, tomando decisiones con y sin LLM según el contexto.
5. El bypass crítico garantiza que las emergencias se manejen en milisegundos, independientemente de la disponibilidad del LLM.

Lecciones aprendidas:

- El LLM es valioso para análisis contextual y consultas en lenguaje natural, pero nunca debe ser el único punto de decisión para eventos de seguridad crítica.
- La latencia de phi3:mini en CPU (1-3 segundos) es aceptable para decisiones no críticas, pero requiere timeouts agresivos y fallbacks.
- El formato JSON forzado no es suficiente: se necesita un extractor robusto que maneje markdown fences y formatos inconsistentes.
- La arquitectura híbrida (determinista + LLM) es la más adecuada para IoT productivo.

Trabajo futuro:

Para la Unidad 3, se integrarán InfluxDB como base de datos temporal, Grafana para visualización avanzada, un motor de predicción analítica, y un agente autónomo mejorado que consuma el estado del gemelo digital para decisiones más contextuales.